



Digital Forensics and Cybersecurity

Malware Analysis Report

ATM Cash Dispenser Malware

Sample SHA-256:

f6609bb3c3197ace26ebdeb372ba657ac84b05a3e9e265b5211e1ea42da70dbe

Prepared by: Abdullah Zia

1. Executive Summary

This report presents a full technical analysis of an ATM cash dispenser malware sample identified by the SHA-256 hash

f6609bb3c3197ace26ebdeb372ba657ac84b05a3e9e265b5211e1ea42da70dbe. The binary is a 32-bit Windows executable compiled with Microsoft Visual C++ 2013, targeting proprietary ATM hardware through a deliberate DLL hijacking mechanism.

The malware impersonates or hijacks a legitimate currency dispenser driver library known as *cscwcng.dll*, which belongs to the CSC (Currency Systems Corporation) hardware interface. Once the target library is loaded, the malware communicates directly with the ATM dispenser hardware to enumerate cassette contents and trigger cash dispensing operations without any legitimate transaction being processed.

Beyond its primary cash theft function, the sample incorporates multiple layers of antianalysis protection including environment checks, sandbox evasion, privilege escalation, and a comprehensive evidence destruction routine that targets USB device registry entries and Windows forensic artifact locations such as Prefetch, Minidump, and Windows Error Reporting directories.

The analysis was conducted in two phases: static analysis using PESTudio, Detect It Easy, and hex inspection, followed by dynamic analysis in a FLARE-VM sandbox environment using Sysinternals Process Monitor with a stub DLL crafted to satisfy the malware's dependency resolution requirements.

2. Static Analysis

2.1 File Hashing and Identification

The first step in any malware analysis workflow is to establish a verified, tamper-evident identity for the sample. This is done by computing cryptographic hash values across multiple algorithms. If even a single byte of the file has been altered, the hash output changes completely, which means matching hashes across multiple algorithms provide strong assurance that the file under analysis is exactly the original sample.

HashCalc was used to compute hashes of the sample across five algorithms simultaneously. The computed values were as follows: MD5

c4234c5a9817a7db447c042aa92f5186,

SHA-1 *a27b8c49ecfb647d1b7eaca7653b271062dd5462*,

SHA-256 *f6609bb3c3197ace26ebdeb372ba657ac84b05a3e9e265b5211e1ea42da70dbe*,

RIPEMD-160 *c532463cde296865bdaee3b4ea8c20c28cbae44f*, and CRC32 *43318530*.

These values collectively serve as the definitive fingerprint of the sample.

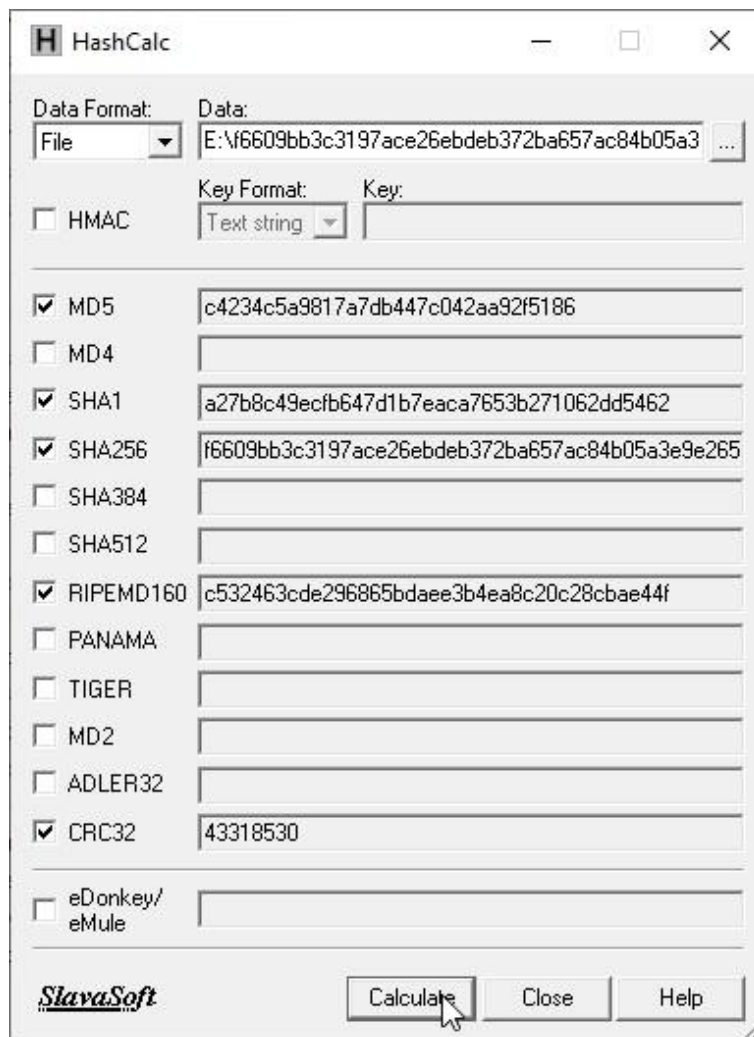


Figure 1: HashCalc output confirming MD5, SHA-1, SHA-256, RIPEMD-160, and CRC32 values for the sample. The SHA-256 value is particularly important for threat intelligence sharing since it is the de facto standard for malware identification in platforms such as VirusTotal. The MD5 and SHA-1 values, while cryptographically weaker, remain widely used in legacy detection systems and incident response workflows.

2.2 PE Header Analysis

The Portable Executable (PE) format is the standard binary format used by Windows executables and libraries. Every PE file begins with a DOS header followed by the PE signature, the COFF file header, the optional header, and a set of section headers. Examining these structures reveals a great deal about how the binary was built and what it is intended to do before a single instruction is executed.

PEStudio was used to perform an initial assessment of the binary's PE metadata. The file is a 32-bit Windows GUI executable with a total size of 100,864 bytes. The compiler timestamp in the PE header records the build date as October 22, 2017 at 15:25:55 UTC. This timestamp can be forged, but in this case it is consistent with the compiler version

identified in later analysis and is treated as likely authentic. The file carries no digital signature, no embedded version strings, no debug symbols, and no overlay data appended after the last section, all of which is consistent with a payload that has been deliberately stripped of identifying information.

One particularly notable field in the manifest embedded within the resources section declares the requested execution level as `asInvoker`. This means that when the binary is launched, it does not request a UAC elevation prompt. An unprepared analyst might interpret this as a sign that the binary has limited capabilities. However, the static strings analysis (discussed in Section 2.5) reveals that the malware explicitly requests `SeTakeOwnershipPrivilege` at runtime through the Windows token API, bypassing the need for a visible elevation prompt by escalating privileges programmatically after launch.

PEStudio flagged 46 indicators of concern on this sample. Among the most significant is a flagged duplicate exports section, which is directly connected to the DLL hijacking mechanism covered in Section 2.5. The import table, on the surface, references only three system DLLs: `KERNEL32.dll`, `USER32.dll`, and `ADVAPI32.dll`. This is an unusually sparse import footprint for a functional application of this size, and it is a strong indicator that additional functionality is resolved at runtime through dynamic API loading rather than static linking.

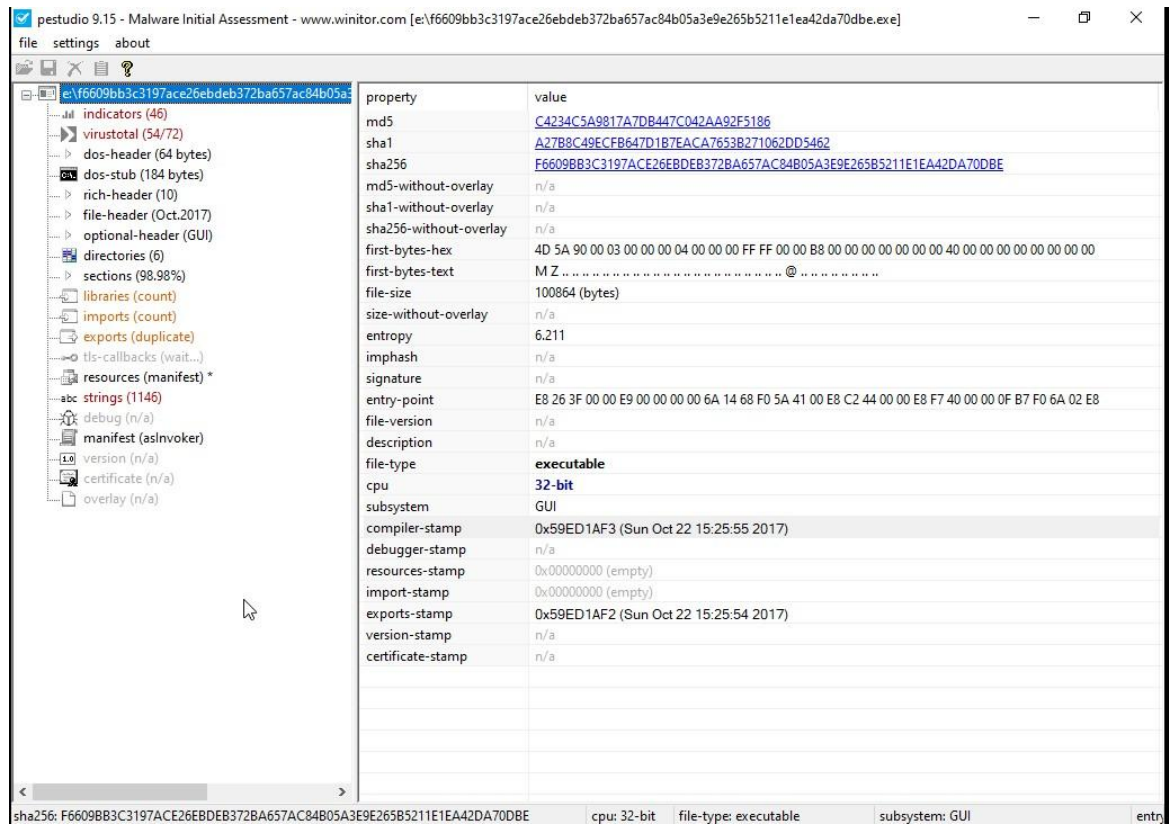


Figure 2: PEStudio initial assessment showing PE metadata, 46 flagged indicators, and key file properties

2.3 Entropy Analysis

Entropy is a measure of randomness or information density within a region of data. In the context of PE analysis, it is used to detect sections of a binary that have been compressed, encrypted, or obfuscated. A fully random or encrypted block of data has entropy close to 8.0, while English text or typical compiled code sits in the range of 4 to 6. When entropy significantly exceeds 6.5 in a code section, it usually means the code has been packed or obfuscated and the true executable content is only revealed at runtime after a decompression or decryption stub runs.

The entropy analysis was performed using the built-in entropy viewer in Detect It Easy. The overall file entropy was measured at 6.211, which is already in a range that warrants attention. However, the more meaningful finding comes from the section-by-section breakdown. The .text section, which contains the executable code, carries an entropy of 6.686 and is explicitly flagged as packed by the analysis tool. This is the most critical section of the file and it occupies the largest contiguous block starting at file offset 0x00000400 with a size of 0x0000F400.

The entropy graph clearly shows a sustained high plateau across approximately the first 60,000 bytes of the file corresponding to the .text section, before dropping sharply as the data and resource sections begin. This flat, elevated plateau is the classic visual signature of a packed code region. By contrast, the .data section registers an entropy of only 2.213, consistent with mostly zero-filled or uninitialised memory. The .reloc section shows moderate elevation at 6.328 which may reflect relocation table density rather than obfuscation.

The practical implication is that static disassembly of the .text section without first running the binary will yield mostly garbage output. The real logic is only accessible after the embedded unpacking stub executes in memory, which is why dynamic analysis is a necessary complement to the static work.

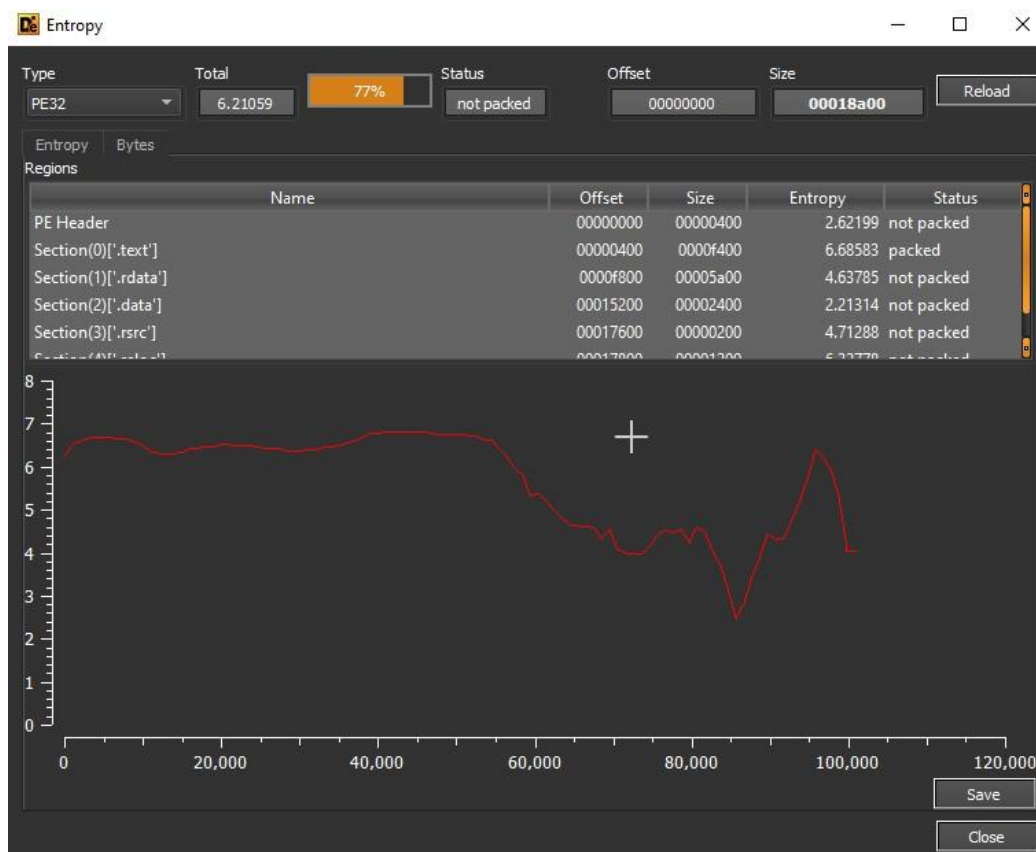


Figure 3: Entropy analysis showing the .text section flagged as packed with entropy 6.686, and the overall entropy plateau across the first 60,000 bytes

2.4 Compiler and Linker Detection

Identifying the compiler and linker used to build a binary helps an analyst understand what development toolchain the malware author used, gives context for the binary's structure and calling conventions, and can sometimes help attribute samples to the same developer if the same unusual build environment appears across multiple samples.

Detect It Easy (DIE) was used to fingerprint the build toolchain. The binary was positively identified as compiled with Microsoft Visual C++ 2013 and linked with Microsoft Linker version 12.0 configured for 32-bit GUI output. The architecture is x86 (Intel i386), littleendian, 32-bit. The entry point sits at virtual address 0x00403fb8 relative to the default base address of 0x00400000. Five PE sections were identified in total.

The fact that this is native Win32 C++ rather than .NET or managed code is significant from an analysis perspective. There is no Common Language Runtime to interrogate, no MSIL to decompile with tools like dnSpy or ILSpy, and no metadata tables to inspect. All analysis of execution logic must be done through x86 disassembly and dynamic tracing.

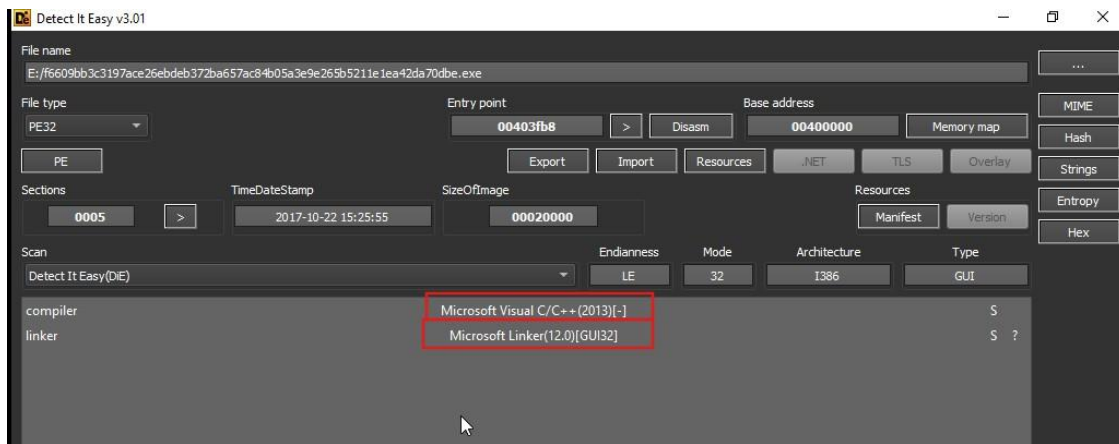
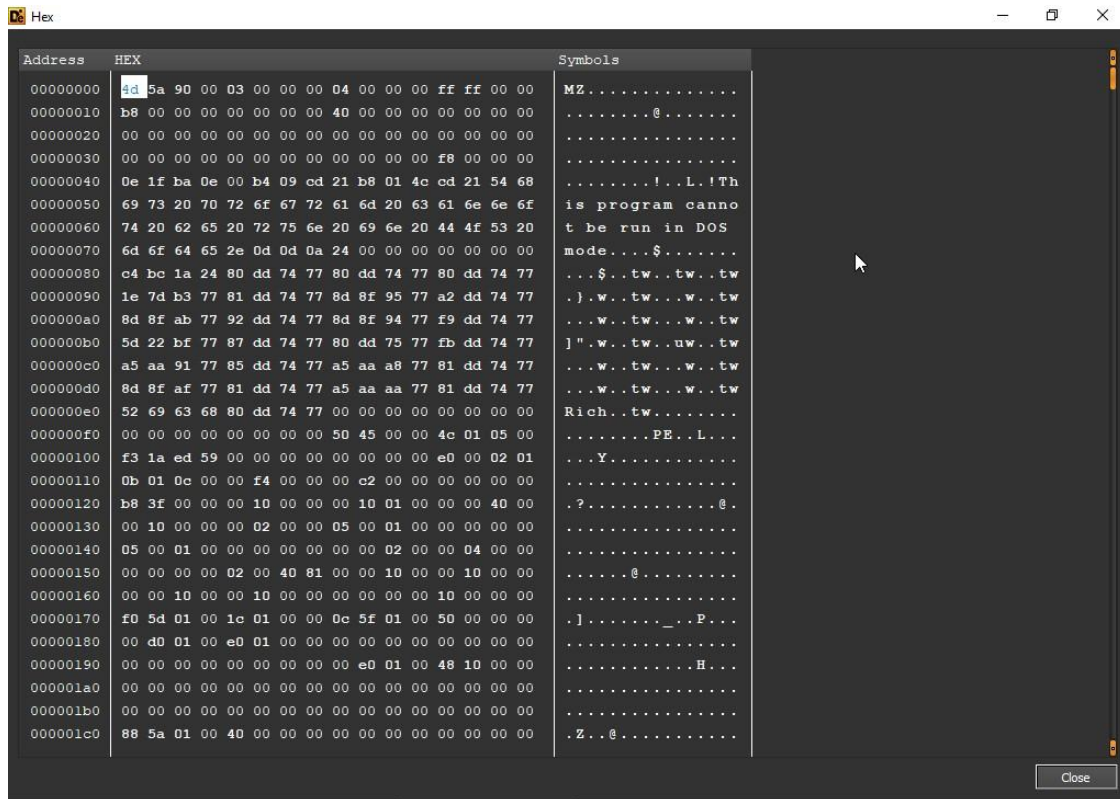


Figure 4: Detect It Easy output confirming Microsoft Visual C++ 2013 compiler, Linker 12.0, x86 architecture, and entry point at 0x00403fb8

2.5 Hex View Inspection

Raw hex inspection of a binary allows an analyst to examine the file at the byte level, independent of any parsing tool that might misinterpret the structure. It is particularly useful for confirming that the PE header is intact, identifying the Rich header (which encodes build tool metadata), and spotting any anomalies in the file structure that higherlevel tools might normalise away.

The hex view at offset 0x00000000 confirms the standard PE file beginning with the bytes 4D 5A, the classic MZ signature named after Mark Zbikowski. The ASCII representation on the right side of the viewer confirms the DOS stub string beginning at around offset 0x0000004D. The Rich header is present starting at approximately offset 0x000000E0 and can be identified by the 52 69 63 68 byte sequence. When decoded, the Rich header reveals the specific compiler product IDs used during the build, which corroborates the DIE findings. The overall header structure appears intact with no evidence of manual patching or corruption.



2.6 Strings Analysis

A total of 1,146 strings were extracted from the sample, a mix of ANSI and Unicode. The strings viewer shows that many of the early entries in the list correspond to section names and internal build artifacts, which is normal. The more operationally significant strings are scattered through the higher-offset regions of the file.

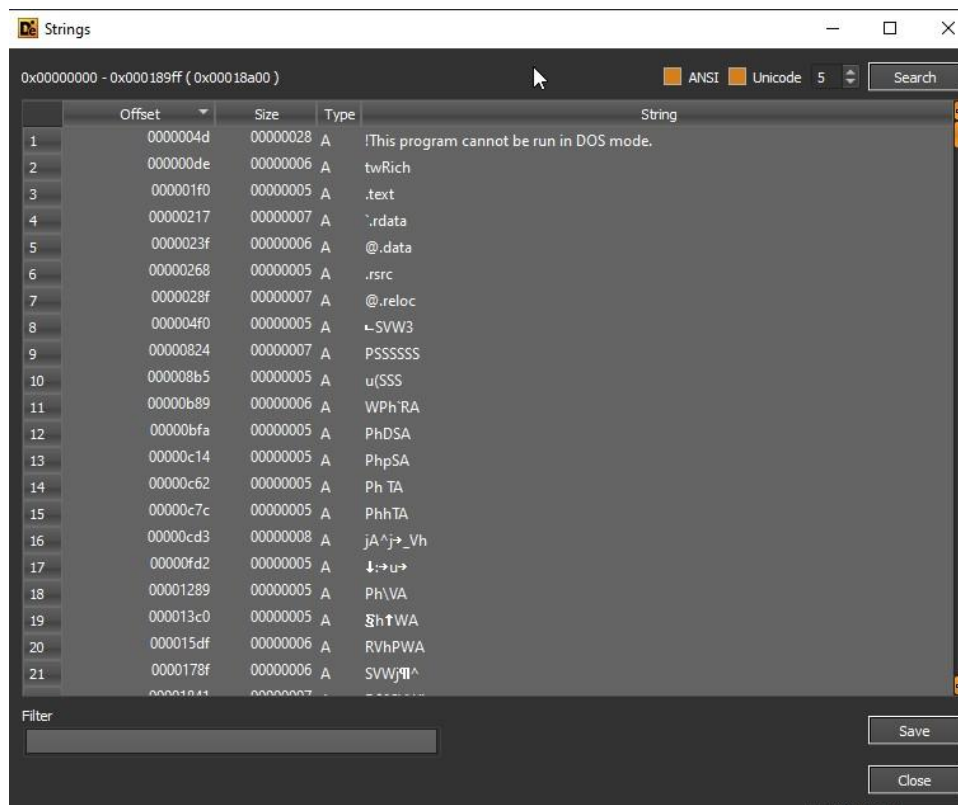


Figure 6: Strings viewer showing initial extraction results including section names and early binary artifacts across 1,146 total strings

DLL Hijacking Mechanism:

The most operationally significant strings in the binary form a complete, sequential DLL search chain that is hardcoded into the executable. The malware tries to load its target library in the following order: first from the default system PATH with no absolute path specified, then from C:\CSCW32\DLL\cscweng.dll, and finally from C:\DLL\cscweng.dll. Corresponding status strings accompany each step of this search, including loading, trying, failed finding, and Loaded success messages, confirming this is a deliberate conditional loading sequence built into the malware's logic rather than incidental import activity.

ATM Hardware Interface:

The binary imports and attempts to resolve eight specific functions from cscweng.dll by name: CscCngOpen, CscCngClose, CscCngLock, CscCngUnlock, CscCngDispense, CscCngTransport, CscCngStatusRead, and CscCngReset. These are proprietary interface functions belonging to a CSC (Currency Systems Corporation) cash dispenser hardware driver. These function names do not appear in any standard Windows system library, which confirms the malware was purpose-built to interact with specific ATM hardware infrastructure rather than being a generic financial malware retooled for ATMs.

Operational strings such as Locking CSCCNG, Unlocking CSCCNG, Dispensing, Dispense failed, Transporting to customer, transport waitpos failed, money not taken, and Finished

dispensing bestCassete with billcount and MaxNotes parameters reveal a complete cash dispensing workflow embedded directly in the binary's logic. The window title string used as a visible label reads "le meme about cutting cookie dough" and serves as deliberate misdirection to avoid drawing attention if the window is briefly visible on screen.

Anti-Forensics and Evidence Destruction:

Several strings confirm that the malware performs active evidence destruction after completing its primary operation. The targeted locations include %SystemRoot%\Prefetch, %Windir%\Minidump, %Windir%\setupapi.log, %Windir%\inf\setupapi.app.log, and %ProgramData%\Microsoft\Windows\WER. The string "Deleting:" followed by a %s%\s format specifier confirms these paths are iterated programmatically and deleted, consistent with the MITRE ATT&CK technique T1070 (Indicator Removal on Host).

Registry Enumeration and Privilege Escalation:

Hardcoded registry paths include

SYSTEM\CurrentControlSet\Enum\USB,

SYSTEM\CurrentControlSet\Enum\USBSTOR,

SYSTEM\CurrentControlSet\Enum\STORAGE\RemovableMedia, and

SYSTEM\CurrentControlSet\Control\DeviceClasses. These paths are used for USB and removable storage enumeration, likely to detect a specific trigger device used to activate the malware or to erase traces of the delivery mechanism. The presence of SeTakeOwnershipPrivilege in the strings confirms the binary requests ownership-level access at runtime, enabled through ADVAPI32 imports including AdjustTokenPrivileges, OpenProcessToken, and SetNamedSecurityInfoW.

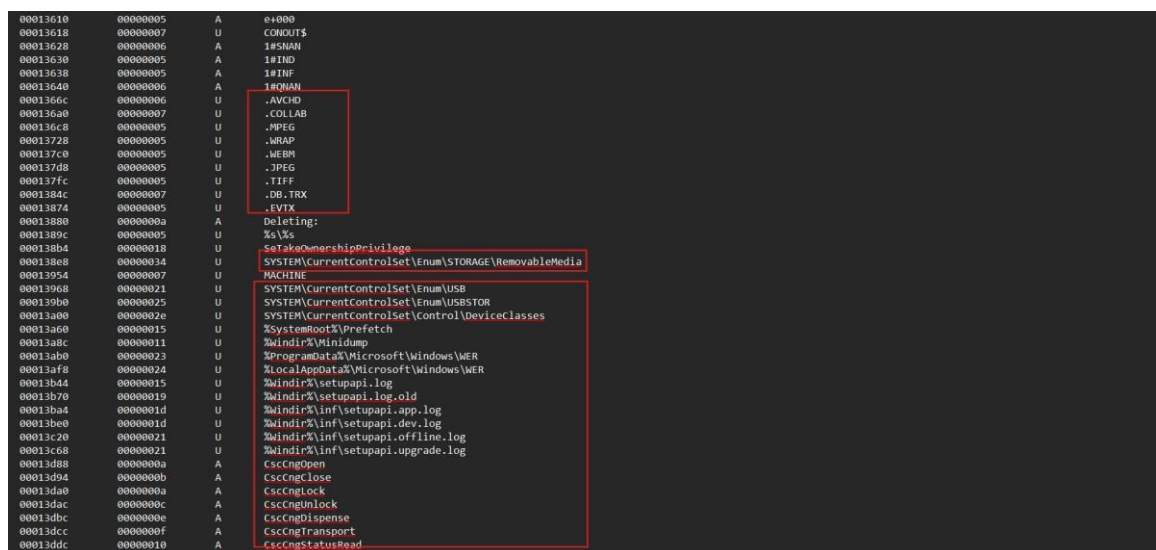


Figure 7: Strings view showing CscCng hardware function names, USB registry paths, forensic artifact deletion targets, SeTakeOwnershipPrivilege, and file extension targets

Anti-Analysis Indicators:

The strings `IsDebuggerPresent`, `GetTickCount64`, and `QueryPerformanceCounter` indicate that the malware checks for the presence of a debugger and measures execution timing, which are standard techniques to detect analysis environments. More unusually, partial strings `5ntel` and `5Genu` were identified, which are fragments of Intel and Genuine as they would appear in the output of a `CPUID` instruction. This suggests the binary performs CPU vendor detection as part of its sandbox evasion logic, since some analysis environments expose virtualised or non-standard CPU vendor strings.

The presence of `LoadLibraryA` and `GetProcAddress` alongside `FreeLibrary` confirms that the malware's minimal static import table is intentional. The bulk of its API surface is resolved dynamically at runtime, making static import analysis insufficient on its own to characterise the full capability of the payload.

3. Dynamic Analysis

3.1 Environment Preparation and Stub DLL Construction

Dynamic analysis requires that the malware be allowed to execute in a controlled environment where its runtime behaviour can be observed. The primary challenge with this sample is that it searches for a proprietary hardware driver library, `cscwcng.dll`, which belongs to CSC cash dispenser hardware and is not available outside of actual ATM infrastructure. If the library is absent, the malware aborts early after failing all three search paths, and there is nothing further to observe.

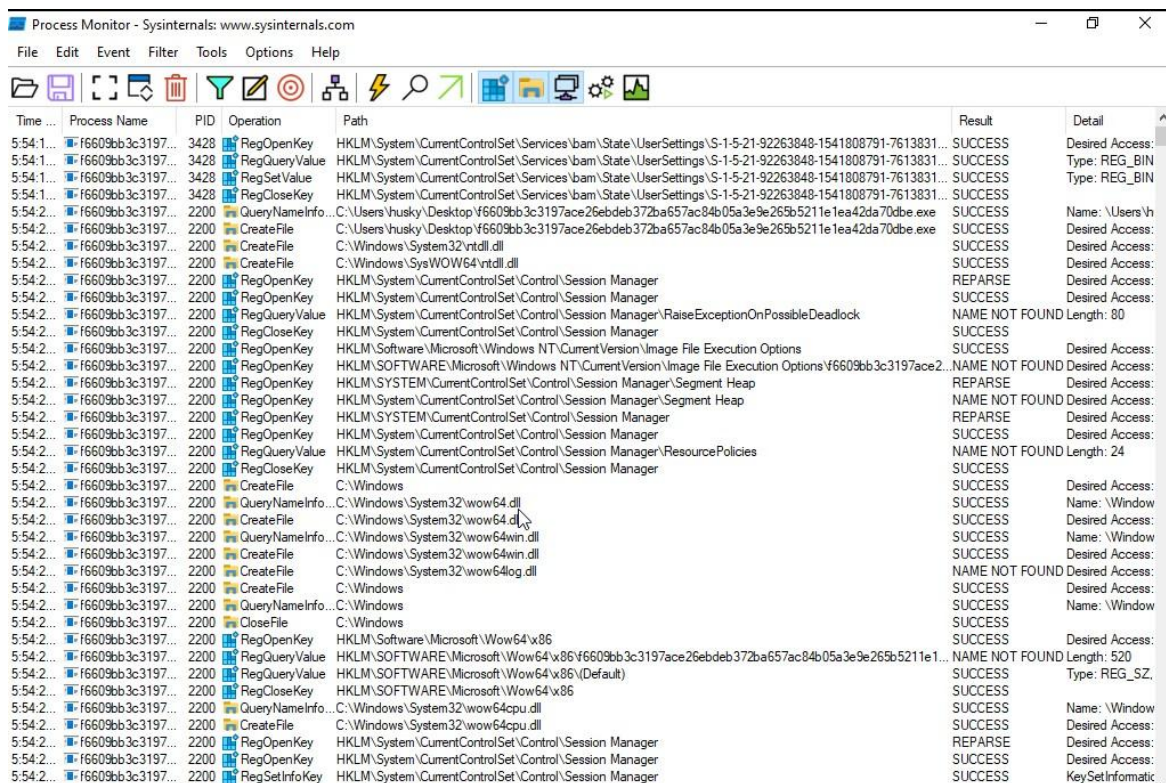
To work around this, a stub DLL was crafted from scratch using the Visual Studio 2017 x86 Native Tools environment on a FLARE-VM analysis machine. The stub was compiled from a C++ source file with a corresponding module definition file to export the required function names. The linker command produced `cscwcng.dll` along with its associated import library and export file. The linker used was Microsoft Incremental Linker version 14.16.27045. After compilation, the exports were verified using `dumpbin`. The stub DLL correctly exported all eight functions the malware attempts to resolve: `CscCngClose`, `CscCngDispense`, `CscCngLock`, `CscCngOpen`, `CscCngReset`, `CscCngStatusRead`, `CscCngTransport`, and `CscCngUnlock`. These function names match exactly what was identified during the static strings analysis. The stub functions themselves are empty stubs that return success codes, which is sufficient to allow the malware's execution to proceed past the dependency resolution stage without crashing.

The stub DLL was placed on the Desktop so that it would be found on the first search path attempt (the default `PATH` resolution before the absolute path fallbacks), and Sysinternals Process Monitor was started in advance to capture all filesystem, registry, and process activity from the moment of execution.

3.2 Process Initialisation and Anti-Analysis Checks

Upon execution, the malware's first observable action was to query the Background Activity Moderator (BAM) registry hive before performing any payload-related operation. BAM is a Windows service that tracks the last execution timestamp of processes. Querying this at startup is not standard application behaviour. It is consistent with the malware checking whether it has been run before on this machine, which would be a technique to prevent repeated execution in monitored or sandbox environments where detonations are triggered multiple times.

Immediately after the BAM check, the process queried the Image File Execution Options registry path for its own image name. IFEO is a mechanism that Windows provides for silently attaching a debugger to a process at launch. The query returned a NAME NOT FOUND result, confirming no debugger had been attached through this mechanism. Together, the BAM check and the IFEO check form the malware's first gate before any payload logic executes.



The screenshot shows the Process Monitor application window with a list of events. The events are filtered to show registry operations and file operations. The first few events are registry queries to the BAM and IFEO paths. The subsequent events show the process creating files and querying the Windows registry for the WOW64 subsystem.

Time ...	Process Name	PID	Operation	Path	Result	Detail
5:54:1...	f6609bb3c3197...	3428	RegOpenKey	HKLM\System\CurrentControlSet\Services\bam\State\UserSettings\S-1-5-21-92263848-1541808791-7613831...	SUCCESS	Desired Access: ...
5:54:1...	f6609bb3c3197...	3428	RegQueryValue	HKLM\System\CurrentControlSet\Services\bam\State\UserSettings\S-1-5-21-92263848-1541808791-7613831...	SUCCESS	Type: REG_BIN
5:54:1...	f6609bb3c3197...	3428	RegSetValue	HKLM\System\CurrentControlSet\Services\bam\State\UserSettings\S-1-5-21-92263848-1541808791-7613831...	SUCCESS	Type: REG_BIN
5:54:1...	f6609bb3c3197...	3428	RegCloseKey	HKLM\System\CurrentControlSet\Services\bam\State\UserSettings\S-1-5-21-92263848-1541808791-7613831...	SUCCESS	
5:54:2...	f6609bb3c3197...	2200	QueryNameInfo...	C:\Users\husky\Desktop\F6609bb3c3197ace26ebdeb372ba657ac84b05a3e9e265b5211e1ea42da70dbe.exe	SUCCESS	Name: \Users\h...
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Users\husky\Desktop\F6609bb3c3197ace26ebdeb372ba657ac84b05a3e9e265b5211e1ea42da70dbe.exe	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Windows\System32\ntdll.dll	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Windows\SysWOW64\ntdll.dll	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	REPARSE	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	REPARSE	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\RaiseExceptionOnPossibleDeadlock	NAME NOT FOUND	Length: 80
5:54:2...	f6609bb3c3197...	2200	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\F6609bb3c3197ace2...	NAME NOT FOUND	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Segment Heap	REPARSE	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager\Segment Heap	NAME NOT FOUND	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\SYSTEM\CurrentControlSet\Control\Session Manager	REPARSE	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegQueryValue	HKLM\System\CurrentControlSet\Control\Session Manager\ResourcePolicies	NAME NOT FOUND	Length: 24
5:54:2...	f6609bb3c3197...	2200	RegCloseKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Windows	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	QueryNameInfo...	C:\Windows\System32\wow64.dll	SUCCESS	Name: \Window
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Windows\System32\wow64.dll	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	QueryNameInfo...	C:\Windows\System32\wow64win.dll	SUCCESS	Name: \Window
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Windows\System32\wow64win.dll	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Windows\System32\wow64cpu.dll	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Windows\System32\wow64log.dll	NAME NOT FOUND	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	QueryNameInfo...	C:\Windows	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	QueryNameInfo...	C:\Windows	SUCCESS	Name: \Window
5:54:2...	f6609bb3c3197...	2200	CloseFile	C:\Windows	SUCCESS	
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\Software\Microsoft\Wow64\X86	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegQueryValue	HKLM\SOFTWARE\Microsoft\Wow64\X86\F6609bb3c3197ace26ebdeb372ba657ac84b05a3e9e265b5211e1...	NAME NOT FOUND	Length: 520
5:54:2...	f6609bb3c3197...	2200	RegQueryValue	HKLM\SOFTWARE\Microsoft\Wow64\X86\{Default}	SUCCESS	Type: REG_SZ
5:54:2...	f6609bb3c3197...	2200	RegCloseKey	HKLM\SOFTWARE\Microsoft\Wow64\X86	SUCCESS	
5:54:2...	f6609bb3c3197...	2200	QueryNameInfo...	C:\Windows\System32\wow64cpu.dll	SUCCESS	Name: \Window
5:54:2...	f6609bb3c3197...	2200	CreateFile	C:\Windows\System32\wow64cpu.dll	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	REPARSE	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegOpenKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	Desired Access: ...
5:54:2...	f6609bb3c3197...	2200	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Session Manager	SUCCESS	KeySetInformatic...

Figure 8: Process Monitor capture showing BAM registry queries, IFEO check, and early WOW64 subsystem initialisation at process startup

3.3 WOW64 Subsystem Initialisation

Because the sample is a 32-bit binary running on a 64-bit Windows host, the Windows-onWindows 64 (WOW64) compatibility layer was invoked during startup. Process Monitor shows the process creating handles to wow64.dll, wow64win.dll, and wow64cpu.dll in

sequence, which are the standard components of the WOW64 environment. This initialisation happened before any payload-related code ran.

The WOW64 context is worth documenting because it has practical implications for analysis. All subsequent API calls and memory operations occurred within the 32-bit WOW64 address space. The DLL search path under WOW64 is slightly different from the native 64-bit search path, and certain registry paths are redirected transparently by the WOW64 subsystem. These redirections can cause confusion if an analyst assumes they are seeing the same registry tree that a native 64-bit process would access.

3.4 DLL Search, Loading, and Memory Mapping

With environment checks passed, the malware began searching for `cscwng.dll`. Process Monitor captured the file operations in detail. The process first opened the DLL file with Read Attributes access to check that it exists and is accessible. It then opened it a second time with Read Data, List Directory, Execute, Traverse, and Synchronize access, which is notably broader than typical DLL loading.

The critical observation is the `CreateFileMapping` operation that followed. The mapping was created with a page protection value of `PAGE_EXECUTE_READWRITE`, which grants simultaneous read, write, and execute access to the mapped memory region. Legitimate DLL loading under the Windows loader uses `PAGE_EXECUTE_READ` or similar read-only protections since there is no reason to write to a DLL's code section after loading. The write permission in this mapping strongly suggests the malware either modifies the loaded code in memory as part of its execution or uses the writable mapping as a staging area for in-memory operations that do not touch the disk.

After the mapping was established, the malware queried the security information and ownership of the loaded module, which is consistent with the `SeTakeOwnershipPrivilege` capability identified in static analysis. The process then proceeded to resolve all eight exported functions from the stub DLL.

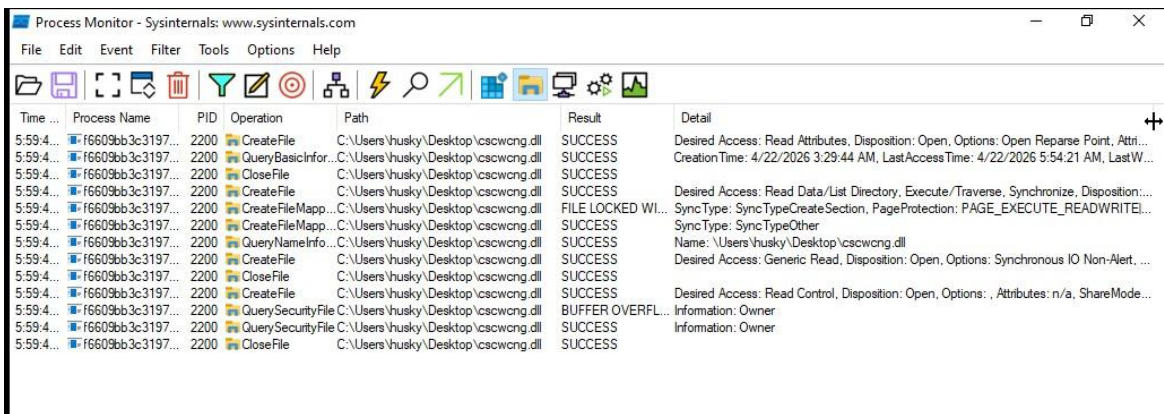


Figure 9: Process Monitor showing the `cscwng.dll` load sequence including `CreateFileMapping` with `PAGE_EXECUTE_READWRITE` protection, indicating potentially malicious in-memory modification

3.5 ATM Hardware Interaction and Cash Dispensing

With the stub DLL loaded and its exports resolved, the malware opened a visible GUI window and began executing the full ATM hardware interaction workflow. The window title was set to "le meme about cutting cookie dough", a nonsensical phrase that was identified during static analysis as a deliberate misdirection string. On a real ATM screen, this title would not be visible to customers, but it would appear in process listings and window manager queries if an operator happened to check the machine's running state.

The window displayed live output from the hardware interaction sequence. The malware queried hardware status, retrieved cassette configuration data, and executed a transport operation. The raw status response visible in the execution window showed a full protocollevel response from the stub driver including cassette parameters across four currency cassettes: cassette 1 holding 1000 EUR denomination notes with 1024 available, cassette 3 holding 200 EUR with 1485 available, and cassette 4 holding 500 EUR with 1371 available. The MaxNotes parameter was set to 20, indicating the maximum number of notes the malware would attempt to dispense in a single operation.

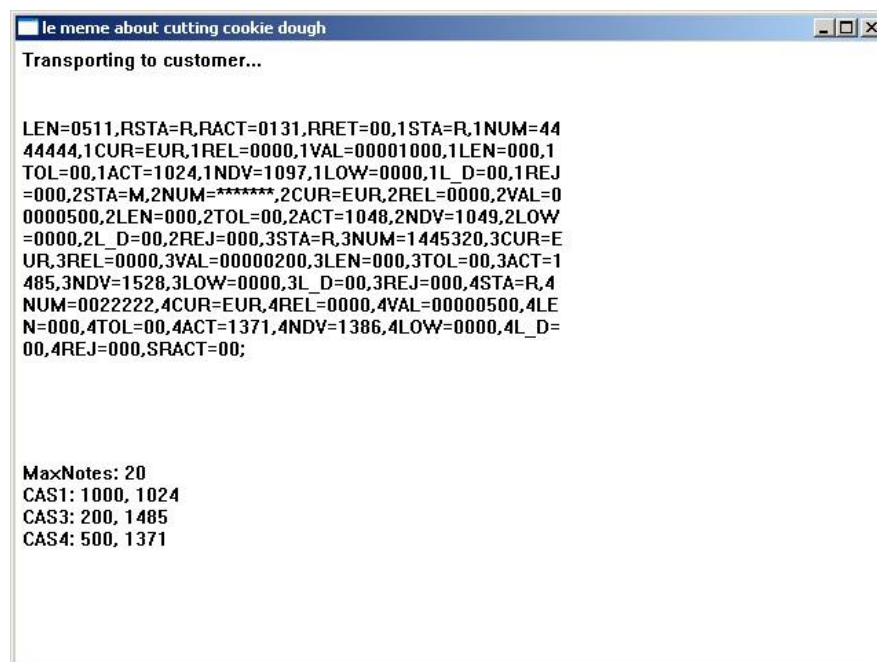


Figure 10: Malware execution window titled "le meme about cutting cookie dough" showing the hardware status response, cassette denomination data, and active transport operation

The sequence of operations observed matches exactly the function call order anticipated from the static strings analysis: open the dispenser, read status, lock, retrieve cassette data, transport to customer position, and dispense. The fact that this sequence executes without any legitimate transaction input or cardholder authentication confirms the malware operates entirely outside of the ATM's normal transaction processing software.

3.6 USB Registry Enumeration and Evidence Destruction

Concurrent with the ATM interaction phase, Process Monitor captured an extensive series of registry deletion operations targeting the USB device enumeration tree. The malware traversed the `HKLM\System\CurrentControlSet\Enum\USB` path and attempted to delete a specific device entry identified by Vendor ID `0xFFFF` and Product ID `0x5678`, a combination that does not correspond to any known commercially manufactured USB device. This strongly suggests the device entry was created specifically by the attacker's delivery mechanism, likely a custom USB drive used to deploy the malware onto the ATM, and is now being erased to remove evidence of how the payload arrived.

The deletion attempts produced a mix of CANNOT DELETE and SUCCESS results. The toplevel device parent key could not be deleted due to system-enforced access controls, but a large number of sub-keys and device property entries including a property key identified by GUID a45c254e-df1c-4efd-8020-67d146a850e0 were successfully removed. This property GUID corresponds to Windows device interface detail properties, and its deletion would remove information that forensic tools use to reconstruct the history of USB devices connected to the machine.

[illegible]

Figure 11: Process Monitor showing mass USB registry deletion operations targeting VID_FFFF&PID_5678 entries, with a mix of CANNOT DELETE and SUCCESS results indicating targeted evidence removal

This behaviour maps directly to MITRE ATT&CK techniques T1070.001 (Clear Windows Event Logs) in spirit and more specifically T1112 (Modify Registry) combined with T1070

(Indicator Removal on Host). The combination of deleting the delivery device record and clearing Windows artifact locations such as Prefetch, Minidump, and WER directories creates a significant gap in the forensic timeline that would make post-incident reconstruction substantially harder.

4. Attack Chain

The complete execution flow of the malware can be summarised in three distinct phases that follow a logical progression from environment validation through payload delivery to evidence destruction.

In the first phase, the malware performs a series of anti-analysis and environment validation checks immediately upon launch. It queries the Background Activity Moderator to detect prior execution, checks Image File Execution Options to confirm no debugger is attached, and performs CPU identification checks using CPUID-derived strings to detect virtualised analysis environments. If any of these checks indicate an analysis environment, the malware exits without proceeding. This gate is what makes sandbox-based automated analysis unreliable for this sample without specific environment preparation.

In the second phase, the malware initialises the WOW64 subsystem for cross-architecture compatibility, escalates its privileges through token manipulation using SeTakeOwnershipPrivilege, and searches for its target library across three hardcoded paths. Once the library is located, it creates a read-write-execute memory mapping of the loaded module and resolves all eight ATM hardware interface functions. It then executes the full dispensing workflow: open, status check, lock, cassette enumeration, transport, and dispense. The entire cash extraction sequence runs without any interaction with the ATM's legitimate transaction processing software.

In the third phase, the malware performs cleanup and evidence destruction. It deletes the USB device registry entries that correspond to the delivery mechanism, removes Windows Prefetch files that would record the binary's execution history, clears Minidump and Windows Error Reporting directories that might contain crash-related telemetry, and purges setup log files that could reveal when and how system changes were made. By the time this phase is complete, the forensic footprint of the attack is substantially reduced.

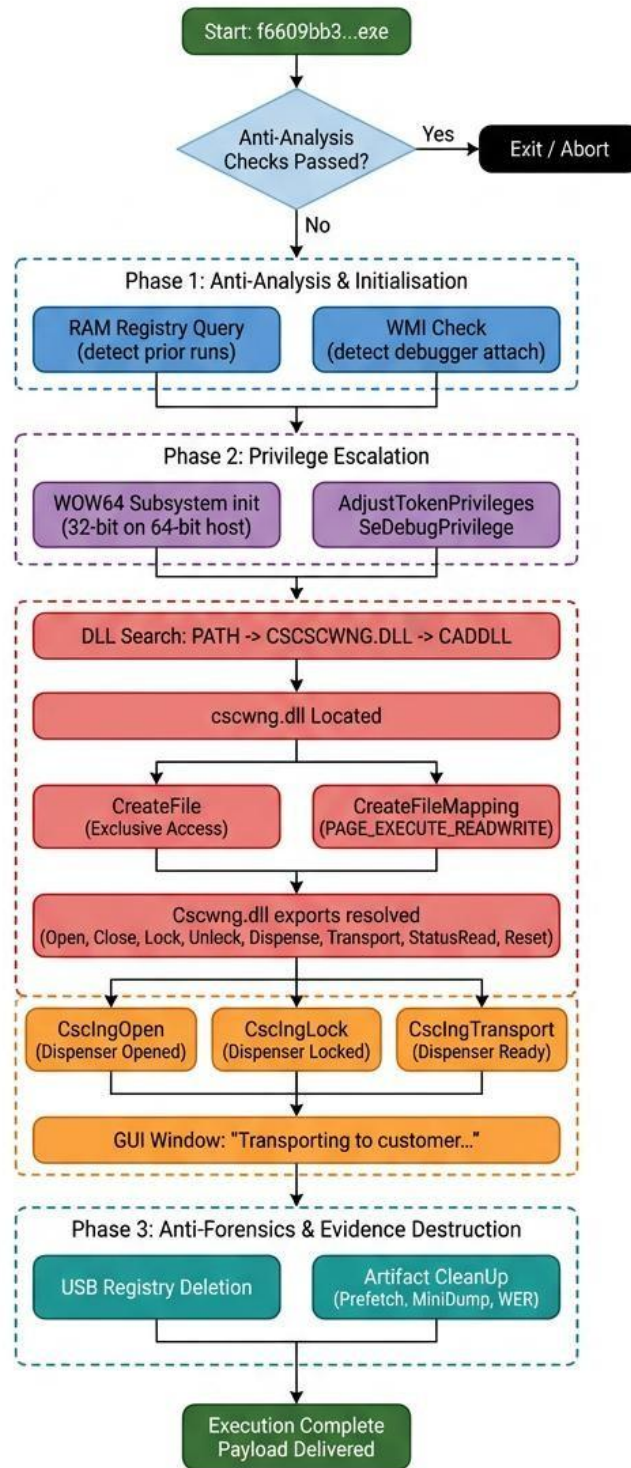


Figure 12: Full attack chain flowchart illustrating the three execution phases: anti-analysis and initialisation, DLL hijacking and cash dispensing, and anti-forensics and evidence destruction

5. Conclusion

This sample represents a sophisticated and purpose-built ATM cash dispenser malware with a well-defined operational lifecycle. It is not a generic banking trojan that has been adapted for ATMs like the function names, the hardcoded DLL search paths, the cassette enumeration logic, and the hardware-level dispensing workflow are all specific to the CSC currency dispenser platform, indicating prior knowledge of the target hardware environment and likely access to documentation or sample hardware during development. The multi-layered anti-analysis capability, covering environment checks, timing-based evasion, CPUID-based virtualisation detection, and evidence destruction, reflects a level of operational tradecraft that goes beyond opportunistic malware. The malware is designed to run once, extract cash, and leave as little trace as possible. The use of a custom USB device with a non-standard vendor and product ID as the delivery vector, combined with subsequent deletion of that device's registry footprint, suggests the attacker had physical access to the ATM prior to the attack and a planned exfiltration mechanism.

From a defensive standpoint, detection opportunities exist at multiple points in the attack chain. Application whitelisting on ATM endpoints would prevent unsigned executables from running. Monitoring for PAGE_EXECUTE_READWRITE memory mappings created by processes that are not the Windows loader is a high-fidelity indicator. Registry monitoring for bulk deletion operations in the USB enumeration tree would catch the cleanup phase. Finally, file integrity monitoring on Prefetch directories and WER folders can detect indicator removal activity even after the fact.

The combination of hardware-specific targeting, anti-analysis depth, and systematic evidence destruction marks this sample as a high-capability, targeted threat against financial infrastructure.